

Understanding the *Spring Framework*

The foundation behind Spring Boot and Java microservices. Before we write any Spring code, let's understand the core concepts: what Spring is, why it exists, and the ideas that power it — IoC, DI, Beans, and the Container.

- Prerequisites: Core Java, Microservices & REST concepts
- No code in this lesson – concepts only
- Next lesson: Spring Ecosystem Overview

01 What is the Spring Framework?

Spring is a **Java framework** — a collection of tools and libraries that makes it easier to build Java applications. It was created in 2003 by Rod Johnson as a simpler alternative to the heavy and complex Java EE (Enterprise Edition) standard.

At its core, Spring does one main thing: it **manages your objects for you**. Instead of you creating and connecting objects manually with `new`, Spring creates them, configures them, and wires them together automatically.



Think of it like hiring a general contractor

When you build a house, you *could* hire the plumber, electrician, painter, and roofer yourself, then coordinate everyone's schedule. Or you hire a **general contractor** who manages everything for you — they know who to hire, when to schedule them, and how to connect the plumbing to the kitchen. **Spring is your general contractor for Java objects.**

 **Big Picture Connection:** Remember our learning path — Microservices (architecture) → REST API (communication) → **Spring Framework** (the engine). Spring is the foundation that powers Spring Boot, which we'll use to actually build our microservices and REST APIs in the next lesson.

A Brief History

2003 — Spring Framework 1.0 released by Rod Johnson. A lightweight alternative to Java EE.

2006 — Spring 2.0 introduced annotation-based configuration. Less XML, more convenience.

2009 — Spring 3.0 added Java-based configuration. No XML needed at all.

2014 — Spring Boot 1.0 released. Made Spring dramatically easier to use.

2017 — Spring 5.0 introduced reactive programming support.

Today — Spring is the most widely used Java framework in the world. Most Java job postings require it.

02 Why Does Spring Exist?

To understand why Spring exists, let's look at the problem it solves.

The Problem: Tightly Coupled Code

In plain Java, when a class needs another class, it creates it directly. This is called **tight coupling** — the two classes are glued together.



The hardwired lamp problem

Imagine a lamp that's **hardwired directly into the wall** — no plug, no socket. It works fine, but if you want to move it to another room, you need to rewire the whole thing. If the bulb breaks, you have to tear apart the lamp to replace it.

Now imagine a lamp with a **standard plug and socket**. You can plug it into any outlet, swap the bulb easily, and even replace the lamp entirely. **Spring gives your Java classes plugs and sockets instead of hardwiring them together.**

Without Spring (Tight Coupling)

If `OrderService` creates `EmailNotificationService` directly inside itself, they're tightly coupled. If you later want to switch to `SmsNotificationService`, you have to open `OrderService` and change its code. Now imagine 50 classes that all create their own dependencies — changing one thing means changing many files.

With Spring (Loose Coupling)

Spring says: *"Don't create your dependencies yourself. Tell me what you need, and I'll give it to you."*

`OrderService` says "I need a `NotificationService`" and Spring provides one. If you switch from Email to SMS, you change the configuration in **one place** — not in every class that uses it.

 **Microservices Connection:** Loose coupling is the fundamental principle of microservices. Each service should be independent and replaceable. Spring enforces this same principle *inside* each service — the classes within your microservice are also loosely coupled, making the code easier to test, change, and maintain.

03 Inversion of Control (IoC)

This is the **most important concept in Spring**. Once you understand IoC, everything else clicks.

Normal Control

In normal Java, **your code is in control**. Your class decides what objects to create, when to create them, and how to connect them. You call the shots.

Inverted Control

With IoC, you **give up that control to Spring**. Spring decides when to create objects, how to configure them, and how to connect them together. You just describe what you need — Spring handles the rest.



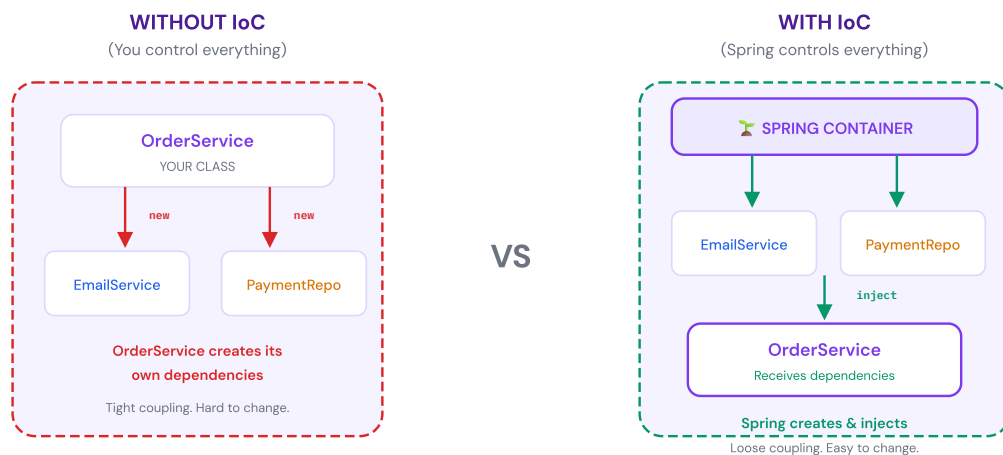
Cooking at home vs ordering food

Normal control (no Spring): You go grocery shopping, pick the ingredients, come home, cook the meal, plate it, and serve it. You control every step.

Inverted control (with Spring): You go to a restaurant, look at the menu, and say "I'll have the pasta." The kitchen handles the ingredients, cooking, plating, and serving. *You described what you wanted, and someone else took control of making it happen.*

That's IoC — **you describe, Spring delivers.**

NORMAL CONTROL VS INVERSION OF CONTROL



Key Takeaway: IoC means you stop saying "I'll create what I need" and start saying "Here's what I need — someone give it to me." That "someone" is the Spring Container.

04 Dependency Injection (DI)

If IoC is the *principle* (give up control), then **Dependency Injection is the technique** Spring uses to implement it. It's the *how* behind the *what*.

What is a Dependency?

A **dependency** is any object that your class needs to do its job. If `OrderService` needs `EmailService` to send confirmation emails, then `EmailService` is a dependency of `OrderService`.

What is Injection?

Injection means Spring *gives* (injects) the dependency to your class, instead of your class creating it. Spring puts the object where it needs to go — you don't go get it yourself.



The IV drip analogy

When you're in a hospital, you don't walk to the pharmacy, pick your medication, and administer it yourself. The nurse **injects** the right medication into your IV drip. You just lie there and receive it.

In Spring: your class is the patient, the dependency is the medication, and Spring is the nurse. **Spring injects the right dependency into your class automatically.**

Three Ways to Inject Dependencies

1

Constructor Injection

Spring passes the dependency through the **class constructor**. This is the **recommended** way — the class can't be created without its dependencies, so nothing is ever missing.

2

Setter Injection

Spring calls a **setter method** on the class to provide the dependency after the object is created. Useful for optional dependencies.

3

Field Injection

Spring sets the field **directly** (using reflection). The shortest to write but harder to test. **Not recommended** for production code.

Which one should you use? Constructor Injection. Almost always. It makes your dependencies explicit, makes testing easy (you can pass mock objects), and guarantees your object is fully ready when it's created. We'll see this in action in the Spring Boot lesson.

 **Microservices Connection:** In a microservice, you'll have many classes: controllers (handle HTTP requests), services (business logic), and repositories (database access). DI is how Spring connects them all. Your controller receives a service, your service receives a repository — all injected automatically. Nobody creates anything with new.

05 What is a Bean?

A **Bean** is simply a Java object that is **created and managed by Spring**. If you let Spring create your object (instead of using `new` yourself), that object is a Bean.

Not every Java object is a Bean — only the ones Spring knows about and manages.



Hotel guests vs visitors

A hotel has many people walking around. But only the ones who **checked in at the front desk** are "guests" — the hotel manages their room, meals, and checkout. Random visitors in the lobby? The hotel doesn't manage them.

In Spring: a **Bean** is an object that "checked in" with Spring. Spring manages its

lifecycle (creation, configuration, destruction). A regular Java object created with `new` is like a visitor — Spring doesn't know it exists.

Key Things About Beans

- **Spring creates them** — you don't use `new`. Spring's container handles creation.
- **Spring manages their lifecycle** — creation, initialization, use, and destruction.
- **Spring injects them** — when another Bean needs this Bean, Spring provides it automatically.
- **Singleton by default** — Spring creates **one instance** and shares it everywhere. If 10 classes need `EmailService`, they all get the **same object**.
- **They live in the Container** — all Beans are stored in Spring's `ApplicationContext` (the Container).

Bean Scopes

SCOPE	WHAT IT MEANS	WHEN TO USE
Singleton (default)	One instance shared across the entire application	Most cases — services, repositories, controllers
Prototype	A new instance every time it's requested	When each use needs a fresh object (rare)
Request	One instance per HTTP request	Web apps — per-request data (e.g., current user context)
Session	One instance per user session	Web apps — user-specific data (e.g., shopping cart)

Don't overthink scopes. In microservices, you'll use Singleton 95% of the time. Your services and repositories are stateless — one shared instance is perfect. You only need other scopes

for special cases.

06 The Spring Container (ApplicationContext)

The **Spring Container** (officially called the `ApplicationContext`) is the heart of Spring. It's the "brain" that does everything we've been talking about:

- **Reads** your configuration (annotations or XML) to know which Beans to create
- **Creates** all the Beans when the application starts
- **Wires** them together (injects dependencies)
- **Stores** them so they can be reused
- **Destroys** them when the application shuts down

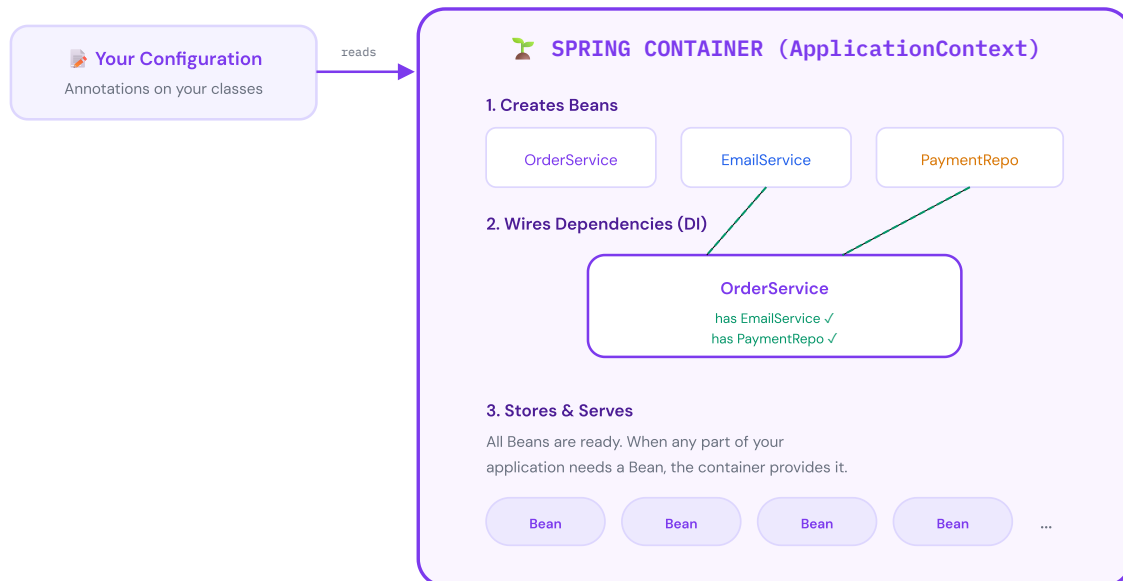


The air traffic control tower

An airport has dozens of planes, ground vehicles, and runways. The **control tower** knows where every plane is, when it should take off or land, and which runway to use. Pilots don't coordinate with each other directly — they all go through the tower.

The Spring Container is your control tower. It knows every Bean, when to create it, and how to connect it to other Beans. Your classes don't find each other — the Container orchestrates everything.

HOW THE SPRING CONTAINER WORKS



 **Microservices Connection:** When you start a Spring Boot microservice, the Container spins up automatically. It creates all your Beans (controllers, services, repositories), wires them together, and starts the embedded web server — all in about 2–3 seconds. Each microservice has its **own** Spring Container managing its own Beans.

07 Spring Modules

Spring is not one giant library — it's organized into **modules**. You pick only what you need. Think of it like a toolbox: you don't carry every tool to every job.



Spring Core

The foundation. Contains IoC, DI, and the Container. Everything else builds on this.



Spring Web (MVC)

For building web applications and REST APIs. Handles HTTP requests, routing, and responses.



Spring Data

Simplifies database access. No more writing boilerplate JDBC code — just define an interface and Spring generates the queries.



Spring Security

Authentication and authorization. Handles login, roles, permissions, and token-based security (JWT).



Spring Cloud

Tools specifically for microservices: service discovery, API gateway, circuit breakers, config server.



Spring Test

Testing support. Makes it easy to write unit tests and integration tests for Spring applications.

 **Microservices Connection:** When we build microservices, we'll combine several of these modules: **Spring Core** (IoC/DI), **Spring Web** (REST endpoints), **Spring Data** (database), and **Spring Cloud** (service discovery, gateway). Spring Boot bundles them all together so you don't have to configure each one manually.

08 Spring vs Spring Boot — What's the Difference?

This is one of the most common questions. Let's clear it up.



The car analogy

Spring Framework = the engine, transmission, and all the mechanical parts. Powerful but complex — you'd need to assemble and configure everything yourself.

Spring Boot = the fully assembled car. Same engine underneath, but it comes pre-

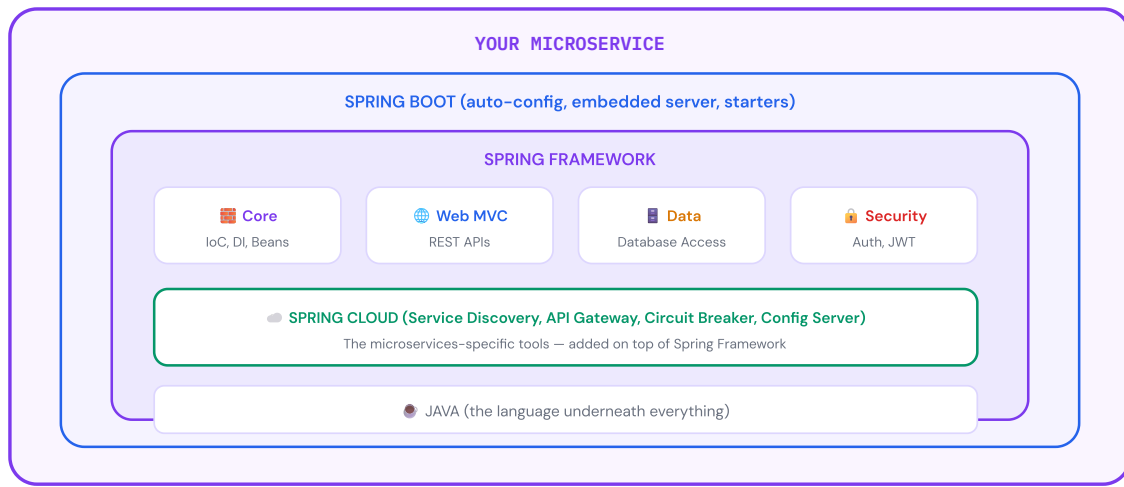
configured, ready to drive. Just turn the key and go.

Spring Boot is not a replacement for Spring — it's Spring **made easy**.

ASPECT	SPRING FRAMEWORK	SPRING BOOT
Configuration	You configure everything manually	Auto-configuration — sensible defaults
Server	You need to set up Tomcat separately	Embedded Tomcat — just run the JAR
Dependencies	You pick each library version manually	"Starters" bundle compatible versions
Getting Started	Hours of setup	Minutes with start.spring.io
Power	Full control over everything	Same power, less boilerplate
Underneath	—	Uses Spring Framework internally

Where Everything Fits

THE SPRING ECOSYSTEM — HOW THE PIECES CONNECT



The learning path we're following: We learned **Microservices** (the architecture) → **REST API** (the communication) → **Spring Framework** (the concepts — today's lesson) → next we'll get an overview of the **Spring Ecosystem** (all the Spring projects and when to use them) → then **Spring Boot** (hands-on building).

09 Summary

All Concepts at a Glance

CONCEPT	WHAT IT IS	ANALOGY
Spring Framework	A Java framework that manages objects and their connections	General contractor for your code
IoC	You give up control of creating objects — Spring does it	Ordering at a restaurant vs cooking at home
DI	The technique — Spring injects dependencies into your classes	Nurse injecting medication via IV drip

CONCEPT	WHAT IT IS	ANALOGY
Bean	A Java object created and managed by Spring	Hotel guest who checked in at the front desk
Container	The brain that creates, wires, and stores all Beans	Air traffic control tower
Spring Boot	Spring made easy — pre-configured, ready to run	Fully assembled car vs loose engine parts



What to take away from this lesson

1. Spring manages your Java objects (Beans) so you don't have to create and wire them manually.
2. IoC = you describe what you need, Spring delivers it. You give up control.
3. DI = the mechanism — Spring injects dependencies through constructors, setters, or fields.
4. A Bean is any object managed by Spring's Container (ApplicationContext).
5. The Container creates all Beans at startup, wires them together, and manages their lifecycle.
6. Spring Boot is Spring made easy — same engine, less configuration.
7. *Next lesson:* We'll explore the **Spring Ecosystem** — all the Spring projects (Spring Boot, Spring Cloud, Spring Batch, etc.) and when to use each one.